Assessment Report

Multivariate Time-Series Prediction Using Deep Learning

Appliance Energy Prediction with the Energy Dataset

Name: Piratheepan M.

Date: 14/10/2026

GitHub repository: https://github.com/Piratheepan22/Appliance_Energy_Forecasting

Table of Contents

1. Introduction 1
   1.1 Objectives 1
   1.2 Summary of the main result 1
2. Data Insights 1
   2.1 Dataset description 1
   2.2 The target variable 2
   2.3 Trends and time patterns 2
   2.4 Correlations 4
   2.5 Memory of the series 4
   2.6 Main observations 4
3. Preprocessing 5
   3.1 Missing values 5
   3.2 Outliers 5
   3.3 Scaling and normalisation 6
   3.4 Data splitting 7
4. Feature Engineering and Selection 8
   4.1 Approach 8
   4.2 Time-based features 8
   4.3 Lag, rolling and history features 8
   4.4 Interaction features 9
   4.5 External and domain features 9
   4.6 Does feature engineering help? 9
   4.7 Feature selection 10
5. Model Design 12
   5.1 Baseline models 12
   5.2 Deep learning architectures considered 12
   5.3 Final architecture and design choices 12
6. Results 14
   6.1 Comparing the architectures 14
   6.2 Final results and comparison with the baselines 15
   6.3 Training behaviour 16
   6.4 Predicted against actual values 16
   6.5 Residual analysis 17
   6.6 Where the model works well and where it struggles 18
   6.7 One-hour-ahead check 19
   6.8 Is the result good? 19

7. Model Optimization 20
7.1 Random search of the hyperparameters 20
7.2 Re-testing the best candidates with new seeds 21
7.3 Dropout sweep 22
7.4 Early stopping, averaging and blending 22
7.5 Effect of the optimisation 23
8. Challenges and Solutions 24
9. Conclusion 24
9.1 Key findings 24
9.2 Limitations 25
10. References 25
Appendix A. Code, Environment and Reproducibility 26

1. Introduction

Energy use of household appliances changes all day long. Being able to predict it a short time ahead is useful for energy management, for example to plan when heavy appliances run or to spot unusual use. This report describes a deep learning model that predicts the energy used by the appliances in a house during the **next 10 minutes**, using the past readings of sensors inside and outside the building.

The work uses the Appliance Energy Prediction dataset (Candanedo, Feldheim and Deramaix, 2017), which is available from the UCI Machine Learning Repository. It contains 19,735 readings taken every 10 minutes between 11 January and 27 May 2016. The target is the column **Appliances**, the energy use of appliances in watt-hours (Wh). All code is in one Google Colab notebook written with TensorFlow/Keras, and every number in this report comes from one complete run of that notebook.

1.1 Objectives

- Explore the data and understand the daily and weekly patterns of energy use.
- Clean the data (missing values, outliers, scaling) without leaking information from the future.
- Create and select features that help prediction, including features based on past energy use.
- Build simple baseline models, then design, train, compare and tune deep learning models.
- Judge the final model on data it has never seen, using MAE, RMSE, MAPE and R-squared.

1.2 Summary of the main result

The final model is a **blend of two CNN-LSTM networks**. A CNN-LSTM first uses a small convolution layer to pick up short patterns, and then an LSTM layer reads the data in time order. The "main" network is trained on raw energy values with Huber loss, and the "companion" network on log-transformed energy values with MSE loss. Each network is the average of five runs with different random seeds, and the two are blended with a weight of 0.65 for the main and 0.35 for the companion network, chosen on validation data only. On the test set (the last 20% of the time line: 3,947 readings from 30 April to 27 May 2016) the final model reached **MAE 23.15 Wh, RMSE 56.52 Wh, MAPE 19.6% and R-squared 0.615**.

This is 13.1% lower MAE and 14.9% lower RMSE than the simple "same as 10 minutes ago" baseline. It is also better than every baseline model on both metrics: 2.1% lower MAE than the best baseline on MAE (Gradient Boosting on log values) and 2.8% lower RMSE than the best baseline on RMSE (Random Forest). The sections below also show where the improvement comes from. Averaging five runs mainly lowers the RMSE, tuning the settings (including the use of Huber loss) mainly lowers the MAE, and the final blending step adds only a small extra gain (about 0.6% to 1.2%) that is similar in size to the normal run-to-run variation.

2. Data Insights

2.1 Dataset description

The file has 19,735 rows and 28 numeric columns plus the date. The time steps are exactly 10 minutes apart with no gaps and no duplicate timestamps, and the data covers 137 days.

Table 1: Columns of the dataset

Group	Columns	Meaning
Target	Appliances	Energy use of appliances in Wh per 10 minutes
Lights	lights	Energy use of lights in Wh
Indoor temperature (°C)	T1 to T9	T1 kitchen, T2 living room, T3 laundry room, T4 office, T5 bathroom, T6 outside (north side), T7 ironing room, T8 teenager room, T9 parents room
Indoor humidity (%)	RH_1 to RH_9	Humidity at the same nine places (RH_6 is outside)
Weather	T_out, RH_out, Press_mm_hg, Windspeed, Visibility, Tdewpoint	From a nearby airport weather station
Random variables	rv1, rv2	Two random numbers

Difference from the assessment brief. The brief lists NSM, WeekStatus and Day_of_week as columns, but this file does not contain them. I created all three from the date column with the same meaning as in the brief (seconds since midnight, weekday or weekend, and day of the week where 0 is Sunday). The file also has columns that the brief does not mention (T7 to T9, RH_7 to RH_9, Tdewpoint, rv1 and rv2).

2.2 The target variable

Energy use is very skewed (Figure 1). The median is 60 Wh and 78.6% of the readings are 100 Wh or lower, but the 99th percentile is 577 Wh and the maximum is 1,080 Wh. The skewness is 3.39 and the kurtosis is 13.67. Most of the time use is low, with short sharp spikes that probably come from cooking, washing or heating water. These spikes are hard to predict, and because errors are squared in RMSE they have a large effect on that metric.

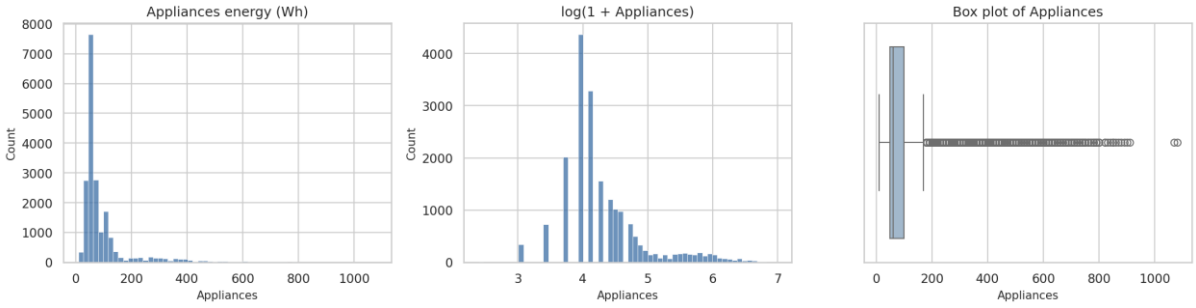


Figure 1: *Distribution of Appliances energy use. The right panel uses log(1 + y) to show the shape of the main body of values.*

2.3 Trends and time patterns

There is no clear long-term trend. Average use stays between 94 and 101 Wh in every month (January 97.0, February 100.9, March 97.0, April 98.9 and May 94.2 Wh), and the 7-day rolling mean moves only a little (Figure 2).

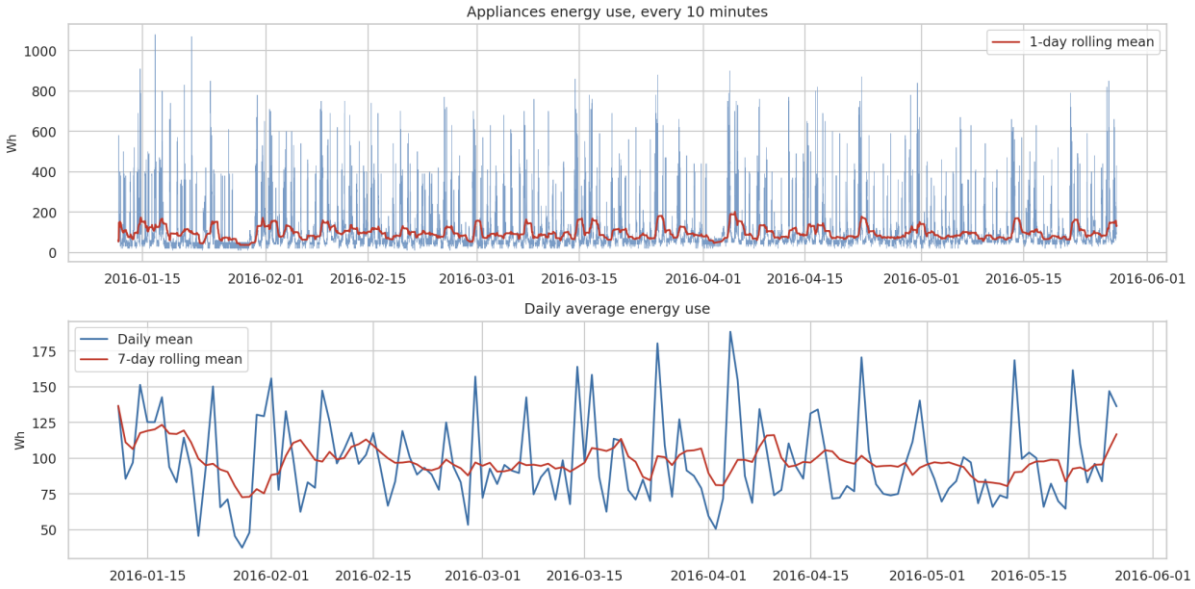


Figure 2: *Appliances energy use over the whole period (top) and daily average with a 7-day rolling mean (bottom).*

There is a clear daily rhythm (Figure 3). Night-time use is lowest at about 50 Wh. It rises from about 07:00, stays around 105 to 135 Wh during the day and reaches its highest level in the evening. Weekday and weekend averages are close (96.6 and 100.6 Wh), but the shape is different: weekends have a stronger late-morning peak (about 160 Wh at 11:00), while weekdays have a higher evening peak (about 195 Wh at 18:00). The heat map shows that the evening peak between 17:00 and 19:00 appears on every day of the week.

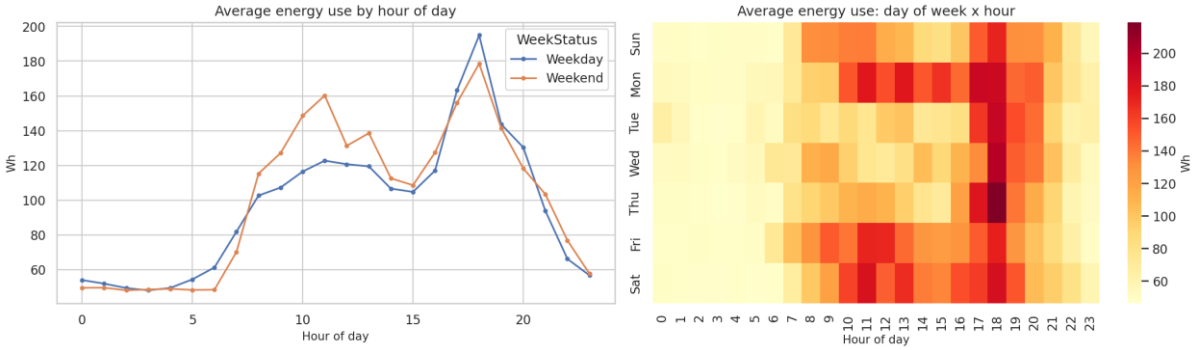


Figure 3: *Average energy use by hour of day for weekdays and weekends (left) and by day of week and hour (right).*

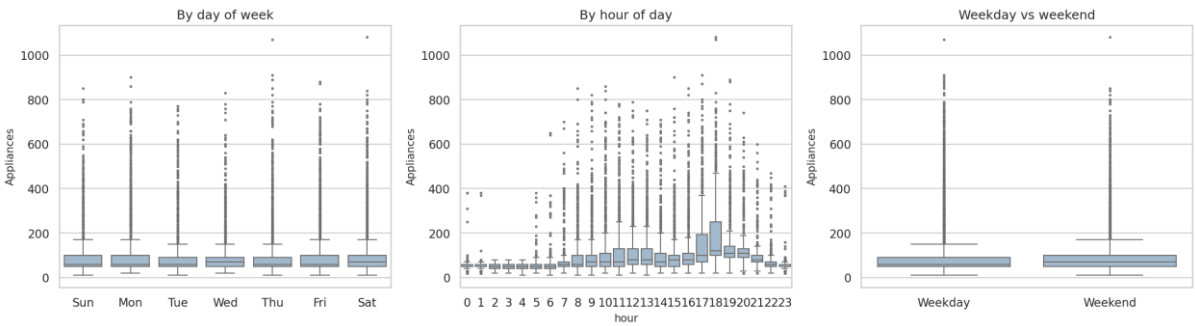


Figure 4: *Box plots of energy use by day of week, hour of day and weekday or weekend.*

2.4 Correlations

Single columns are only weakly related to the target (Figure 5). The strongest positive correlation is with lights (0.20). The indoor sensors T2 and T6 and the outdoor temperature have correlations of about 0.10 to 0.12, and outdoor humidity is slightly negative (-0.15). The random variable rv1 has a correlation close to zero, as expected. Many temperature columns are strongly related to each other (the rooms warm and cool together), so they carry overlapping information. Because no single sensor is strongly related to the target, a model that only uses sensor values cannot do well, and we expect recent energy use to be the most useful input.

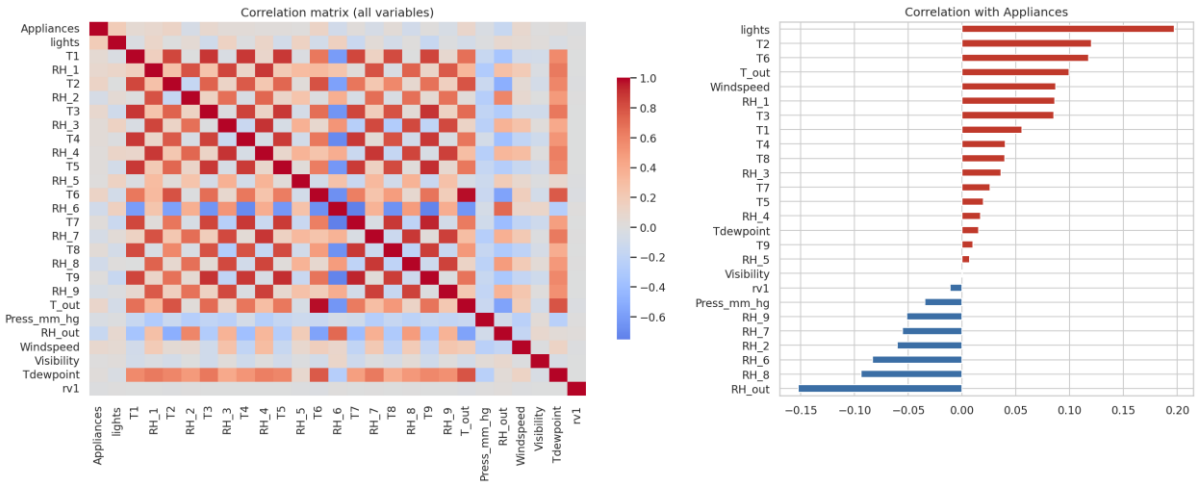


Figure 5: *Correlation matrix of all variables (left) and correlation of each variable with Appliances (right).*

2.5 Memory of the series

The autocorrelation of Appliances, measured on the training period only, is 0.76 at a lag of 10 minutes, 0.54 at 20 minutes, 0.44 at 30 minutes and 0.32 at one hour. A small daily echo remains at one day (0.21). The partial autocorrelation is significant for lags 1 to 10, 12, 21 and 24 (Figure 6). So the series has a short memory of roughly one to four hours, and this guides the choice of lag features in Section 4.

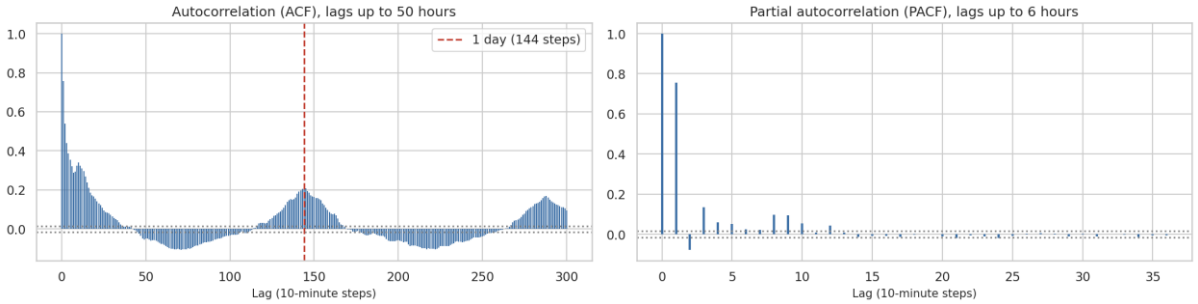


Figure 6: *Autocorrelation (left) and partial autocorrelation (right) of Appliances in the training period.*

2.6 Main observations

- Energy use is low most of the time with short, large spikes; the evening and the late morning are the busiest times.
- There is no trend over the months, so the level of use is stable.
- Day of week matters less than hour of day.

- Sensor values alone are weak predictors; the recent history of the target is the most promising input.

3. Preprocessing

3.1 Missing values

I checked for missing values and also for missing time steps (a missing row would not show up as an empty cell). There are **no missing values and no missing 10-minute steps**.

Because nothing needed filling, I tested our planned method on purpose. I hid 5% of known values, either as single readings or as 1-hour blocks, and measured how well three methods could recover them. Linear interpolation over time was clearly the best for the smooth sensor and weather columns (Table 2). For example, for outdoor humidity the error of single missing readings was 0.23 with interpolation, against 0.97 with forward fill and 14.06 with the mean. For Appliances, which is spiky, no method is good, but interpolation is still the least bad.

Table 2: Error (RMSE) when hidden values are filled by three methods (lower is better)

Gap type	Column	Forward fill	Linear interpolation	Mean fill
10 minutes	T1	0.052	0.028	1.625
10 minutes	T_out	0.177	0.037	4.959
10 minutes	RH_1	0.580	0.313	3.921
10 minutes	RH_out	0.969	0.234	14.059
10 minutes	Appliances	89.626	60.862	110.148
1-hour blocks	T1	0.156	0.053	1.605
1-hour blocks	T_out	0.708	0.156	5.245
1-hour blocks	RH_1	1.540	1.209	4.251
1-hour blocks	RH_out	3.338	0.944	15.715
1-hour blocks	Appliances	118.722	91.225	111.209

Decision. The pipeline contains a cleaning function that puts the data on a regular 10-minute grid and fills short gaps (up to one hour) with time-based linear interpolation. On this dataset it changes no values. The mean is not used, because it ignores the time order and would create artificial flat values.

3.2 Outliers

I counted outliers with two common rules: the IQR rule (outside 1.5 times the inter-quartile range from the quartiles) and the Z-score rule (more than 3 standard deviations from the mean). The columns with the most outliers are lights, Visibility, Appliances and RH_5 (Table 3, Figure 7).

Table 3: Columns with the most outliers

Column	IQR outliers	IQR %	Z-score > 3	Z-score %
lights	4,483	22.72	647	3.28
Visibility	2,522	12.78	37	0.19
Appliances	2,138	10.83	540	2.74
RH_5	1,330	6.74	567	2.87
T2	546	2.77	206	1.04
T1	515	2.61	14	0.07
T6	515	2.61	98	0.50

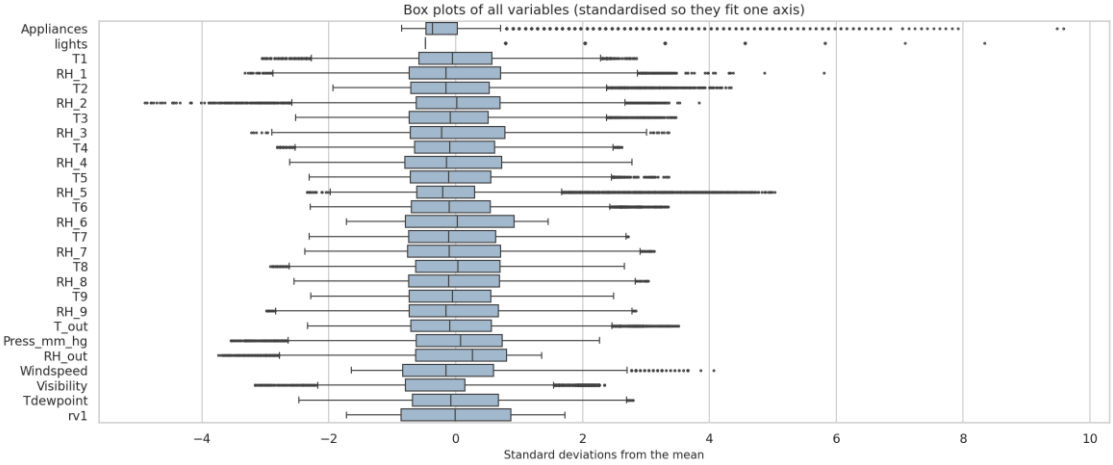


Figure 7: *Box plots of all variables, standardized so that they share one axis.*

Decisions and reasons

1. **The target is not changed.** The high readings of Appliances (up to 1,080 Wh) are real events and not sensor errors. Removing them would make the scores look better but would make the model useless for the spikes that also appear in the test data.
2. **Extreme sensor and weather values are capped.** For each sensor and weather column we cap values below Q1 - 3 x IQR or above Q3 + 3 x IQR, with the limits learned only from the training period. This changed 803 of 473,640 values (0.17%): 421 in the training period and 382 in the test period. Most were in RH_5 (514) and T2 (260). No rows are deleted.
3. **Lights and RH_6 are left as they are.** Lights is mostly zero with real on-events. RH_6 (outdoor humidity) sits at its lower limit of 1.0 for 1,452 readings (7.4%), which looks like a sensor floor in very dry air; capping does not change these values.

I first tried simple percentile limits (0.5th and 99.5th percentile of the training period). This capped about 22,000 values in the test period, because May is warmer than January to April and the test temperatures naturally lie above the training range. Cutting them would have removed real information, so I changed to the wider 3 x IQR limits (see Section 8).

3.3 Scaling and normalization

I use **standardization** (subtract the training mean and divide by the training standard deviation) for all input features. After the extreme values are capped, most features are roughly bell-shaped or bounded, and standardization is less affected by a few extreme values than Min-Max scaling, which

squeezes most values into a small range when one value is far away. It also suits the tanh and sigmoid gates inside LSTM layers. The scaler is fitted on the training rows only and then applied to the validation and test rows, so no test information reaches the model.

The target is also standardized for training. I train some models on a second version, log(1 + y) followed by standardization, because the target is skewed (Sections 5 and 7). Predictions are always converted back to Wh before MAE and RMSE are computed, so all reported errors are in the original unit.

3.4 Data splitting

Because the data is a time series, it is never shuffled before the split. The first 80% of the time line is the training period and the last 20% is the test set. The last 15% of the training period is kept as a validation set, which is used to choose features, architecture and settings, and for early stopping. The test set is used only once, at the end (Table 4, Figure 8).

Table 4: Chronological split (rows are model inputs with a valid target)

Set	Rows	Share	From	To
Training	13,296	67.9%	12 Jan 2016 17:00	14 Apr 2016 00:50
Validation	2,346	12.0%	14 Apr 2016 01:10	30 Apr 2016 08:00
Test	3,947	20.1%	30 Apr 2016 08:10	27 May 2016 17:50

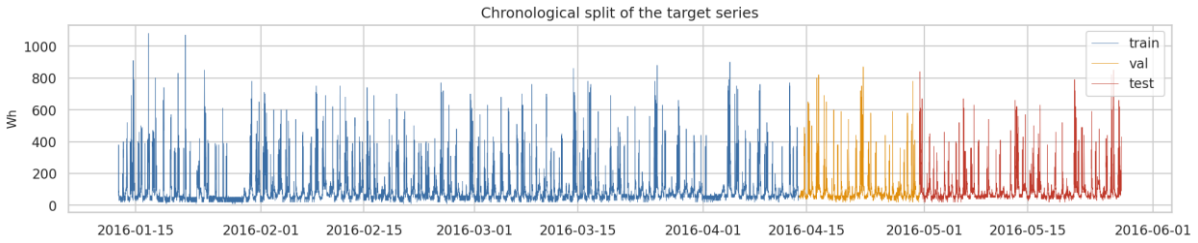


Figure 8: *The target series colored by training, validation and test period.*

Steps taken to avoid data leakage:

- A row belongs to a set according to the time of the value it predicts. No training row has a label inside the test period, and a small gap separates the training and validation rows.
- The scaler, the outlier limits, the lag selection and the feature selection all use training data only.
- Every engineered feature uses only current or past values. A test in the notebook changes all data after one point in time and checks that the features at that point stay exactly the same (it passed).
- Windows for the test set may look back into the validation and training period. This is allowed, because it is past information that is available at prediction time.

4. Feature Engineering and Selection

4.1 Approach

Each row of the model input describes the situation at time t, and the target is the energy use at time t + 10 minutes. Every feature uses only information that is available at time t or earlier. I built 93 candidate features in five groups (Table 5) and then selected the useful ones.

Table 5: Candidate features

Group	Count	Features	Why they may help
Original sensors	25	lights, T1-T9, RH_1-RH_9, T_out, RH_out, Press_mm_hg, Windspeed, Visibility, Tdewpoint	The raw measurements of the house and the weather
Calendar	12	NSM, hour, day of week, weekend flag, holiday flag, day-off flag, sine and cosine of time of day and of day of week, night flag, evening-peak flag	Daily and weekly rhythm seen in the EDA
Recent history	36	Current value, 16 lags, rolling mean (30 min, 1 h, 3 h, 6 h, 24 h), rolling std, max and min (1 h, 3 h), two exponential averages, change since the last reading, level compared with the 24-hour average, share of the last 3 hours above 100 Wh, steps since the last spike, typical use at the target time, current level compared with the typical level	The series has a short memory
Rolling context	3	Rolling mean of lights (1 h), outdoor temperature (3 h) and wind speed (1 h)	Smooths noisy signals
Interactions	17	Mean indoor temperature and humidity, indoor-outdoor differences, temperature x humidity, wind x temperature difference, kitchen minus living room differences, and 10-minute, 30-minute and 1-hour changes of humidity and temperature in the kitchen, laundry room and bathroom	Cooking, showers and laundry change humidity quickly; heating depends on the indoor-outdoor gap

4.2 Time-based features

I extracted the hour of day, seconds since midnight (NSM), day of week (0 = Sunday), a weekend flag, a holiday flag and a "day off" flag (weekend or holiday). Because 23:50 and 00:00 are close in time but far apart as numbers, I also encode the time of day and the day of week with a sine and a cosine. These features describe the **moment being predicted**, which is safe because the calendar is known in advance.

The month is **not** used as a model input. The test period is in May (and the last day of April), a month the model never sees in training, so a month feature could not be learned in a useful way. Month is only used in the EDA.

4.3 Lag, rolling and history features

Lags chosen from autocorrelation. The partial autocorrelation (Section 2.5, training period only) is significant for lags 1 to 10, 12, 21 and 24. I used all of these, plus 36, 72 and 144 steps (6 hours, 12 hours and one day) to cover slower patterns, giving 16 lags from 10 minutes to one day.

Rolling statistics. I computed the rolling mean over 30 minutes, 1 hour, 3 hours, 6 hours and 24 hours, the rolling standard deviation, maximum and minimum over 1 hour and 3 hours, and two

exponentially weighted averages. These smooth the noisy signal and describe how busy the recent past was. All windows end at the current time, so no future value is used.

Features designed for this problem.

- **Steps since the last spike:** the number of 10-minute steps since the energy use was last above 100 Wh (capped at one day). It tells the model whether a busy period has just ended or whether the house has been quiet for a long time.
- **Typical use at the target time:** the average energy use at the same clock time on earlier days of the same type (working day or day off). It gives the model the usual pattern for the moment being predicted without using any future value. If a day type has no history yet, the average over all day types is used.
- **Level ratio and level against the usual level:** the current reading divided by the 24-hour average, and the difference between the (log) current reading and the (log) typical use at the target time. They tell the model whether the house is more active or quieter than usual right now.
- **Momentum and active share:** the change since the last reading, and the share of the last 3 hours in which the use was above 100 Wh.

4.4 Interaction features

I created features that combine sensors: the mean indoor temperature and humidity (T6 and RH_6 are outside, so they are excluded), the indoor-minus-outdoor differences of temperature and humidity, temperature multiplied by humidity, wind speed multiplied by the indoor-outdoor temperature difference (a heat-loss indicator), and the differences between the kitchen and the living room. I also added short-term changes (over 10 minutes, 30 minutes and one hour) of kitchen, laundry-room and bathroom humidity and kitchen temperature, because cooking, washing and showers raise humidity quickly.

4.5 External and domain features

The brief suggests external factors such as holidays "if applicable". I added a holiday flag with the four Belgian public holidays inside the data period (28 March, 1 May, 5 May and 16 May 2016), because the weather data comes from a Belgian airport station. A simple check showed no consistent holiday effect: the daily average on the holidays differed from normal days of the same weekday by between about -11 and +17 Wh, which is inside the normal day-to-day variation of about 30 Wh. In addition, only one of the four holidays falls in the training period. The feature-selection step did not keep the flag (Section 4.7). The result is still useful: it shows that this household's routine does not change much on public holidays.

4.6 Does feature engineering help?

To check the value of each group, I added the groups one by one and trained a Random Forest on the training set each time, judged on the validation set (Table 6). Sensors alone cannot predict the target: the R-squared is negative, which means worse than always predicting the average. Calendar features help a little. The large improvement comes from the **recent history of the target**, which brings the validation MAE from about 60 Wh down to 28 Wh. The context and interaction features

did not give a clear extra gain on top of the history features (RMSE 58.30 against 58.67 Wh, but MAE 27.97 against 27.90 Wh).

Table 6: Random Forest on the validation set, adding feature groups step by step

Feature set	Features	MAE (Wh)	RMSE (Wh)	R-squared
Original sensors only	25	73.76	111.61	-0.334
+ calendar features	37	60.02	101.15	-0.096
+ recent history (lags, rolling)	73	27.90	58.67	0.631
+ context and interactions (all)	93	27.97	58.30	0.636

4.7 Feature selection

Many of the 93 candidates are similar to each other, and too many overlapping inputs make a neural network noisier and slower. I selected features in three steps, using only the training rows (and the validation rows to judge the result):

1. **Correlation filter.** If two features have a correlation above 0.98, only one is kept. This removed three features (hour, the 3-hour rolling maximum and the 1-hour rolling wind speed), leaving 90.
2. **Noise benchmark.** I added two random columns (rv1 and our own random column) and trained a Random Forest. A feature that is not more important than the best random column is not useful. Only 23 of the 90 features passed.
3. **Recursive Feature Elimination (RFE).** A Random Forest repeatedly removes its least important features to create a ranking.

I then compared several feature sets on the validation set (Table 7). The score used here is the one used for all decisions in this report: MAE and RMSE are each divided by the error of the persistence baseline on the same data and then averaged, so 1.0 means "as good as persistence" and lower is better. The rule was: take the smallest set whose score is within 1% of the best. The chosen set is the **top 15 features from RFE**. Its score (0.874) is almost the same as the best set (0.868, the 23 features above the noise level), while the set with all 93 candidates is clearly worse (0.924). More features did not help; they added noise.

Table 7: Comparison of feature sets (Random Forest, validation set)

Feature set	Features	MAE (Wh)	RMSE (Wh)	Score
All candidates	93	27.97	58.30	0.924
After correlation filter	90	28.19	58.46	0.929
Above noise level	23	25.50	56.80	0.868
RFE top 20	20	25.62	57.32	0.874
RFE top 15 (chosen)	15	25.69	57.15	0.874
RFE top 10	10	26.15	58.40	0.892

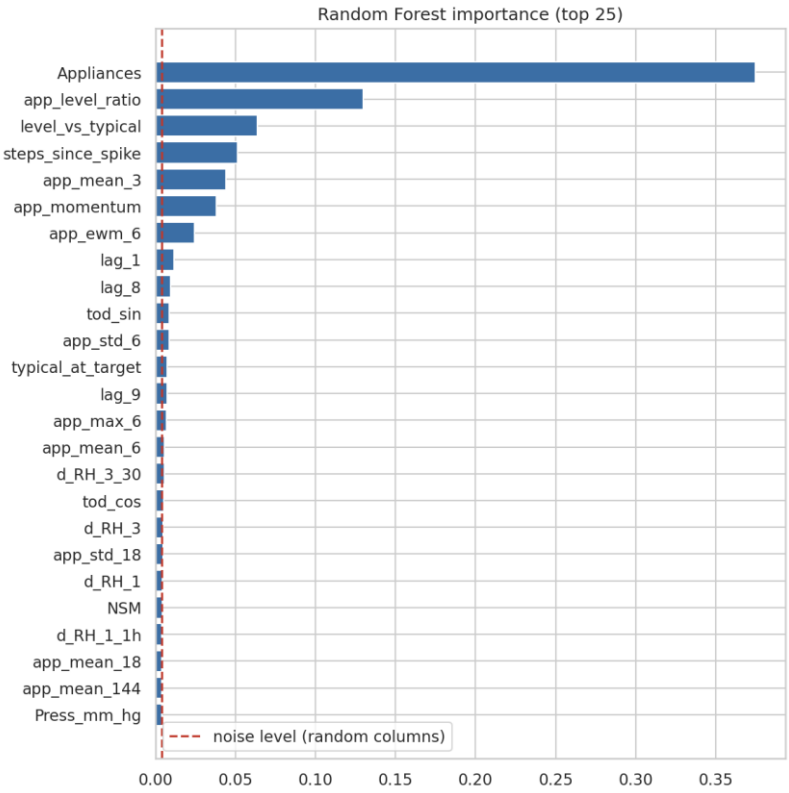


Figure 9: *Random Forest importance of the top 25 features. The dashed line is the importance of the best random column.*

Table 8: The 15 selected features

Feature	Group	Meaning
Appliances	History	Energy use now (lag 0)
lag_1	History	Energy use 10 minutes ago
lag_8, lag_9	History	Energy use 80 and 90 minutes ago
app_mean_3	History	Average of the last 30 minutes
app_mean_6	History	Average of the last hour
app_ewm_6	History	Exponentially weighted average, about the last hour
app_std_18	History	Variation of energy use over the last 3 hours
app_momentum	History	Change since the last reading
app_level_ratio	History	Current use divided by the 24-hour average
level_vs_typical	History	Current level compared with the usual level at the predicted clock time
steps_since_spike	History	Time since the last reading above 100 Wh
typical_at_target	History	Usual energy use at the predicted clock time on earlier days of the same type
NSM, tod_sin	Calendar	Seconds since midnight, and the time of day as a sine

What the selection tells us. Thirteen of the fifteen features describe the recent history of the target and two describe the time of day. No raw sensor, no weather variable, no holiday flag, no rolling context feature and no interaction feature was kept. For a forecast only 10 minutes ahead, the current and recent energy use already contain almost everything the sensors could add. Three of the new history features (level ratio, level against the usual level, and momentum) were kept, which

suggests that describing the current activity relative to the household's normal level is more useful than the raw sensor values. The sensors may become more useful for longer horizons, but the one-hour check (Section 6.7) did not test this separately.

The feature table, the selected list and the scaler are saved in the repository so that the same inputs can be rebuilt.

5. Model Design

5.1 Baseline models

Baselines show what "good" looks like. If a deep model cannot beat a simple one, its extra complexity is not worth it. All baselines use the same selected features and the same training, validation and test rows.

- **Persistence:** the next 10 minutes will equal the current reading. This is the standard "do nothing smart" benchmark for forecasting.
- **Moving average (1 hour):** the average of the last six readings.
- **Linear Regression** (required baseline).
- **Random Forest** with 200 trees.
- **Gradient Boosting** (an extra, stronger tree-based reference).
- **Random Forest and Gradient Boosting on a log-transformed target.** The neural networks are also trained on log values, so the tree models are given the same option. This keeps the comparison fair.

5.2 Deep learning architectures considered

The design follows common practice described by Chollet (2021) and in the TensorFlow time-series forecasting tutorial. A sliding window of the past is given to each network. For every prediction the model sees the last W time steps of the 15 selected features and predicts the energy use 10 minutes after the last step. I compared four architectures with the same default settings (Section 6.1).

Table 9: Architectures compared

Architecture	Idea
MLP	Flattens the window and uses ordinary dense layers. It shows whether modelling the time order is needed at all.
LSTM	Recurrent layers with memory gates (Hochreiter and Schmidhuber, 1997); good at keeping information over time.
GRU	A lighter version of the LSTM with fewer parameters (Cho et al., 2014); often as good on small data.
CNN-LSTM	A 1-D convolution first finds short local patterns (for example a quick rise in humidity), then LSTM layers read them in time order.

5.3 Final architecture and design choices

The architecture with the best validation score was the CNN-LSTM. The final model uses two CNN-LSTM networks with different settings, both found by the optimization steps in Section 7: a main

network (raw target) and a companion network (log target). Their layers are shown in Tables 10 and 11. The main network is very small (10,337 trainable parameters) and the companion network is larger (271,137).

Table 10: Main network (raw target, Huber loss)

Layer	Settings	Output shape	Parameters
Input	24 time steps (4 hours) x 15 features	(24, 15)	0
Conv1D	32 filters, kernel size 3, causal padding, ReLU	(24, 32)	1,472
Dropout	rate 0.1	(24, 32)	0
LSTM	32 units, tanh, returns the last step only	(32)	8,320
Dropout	rate 0.1	(32)	0
Dense	16 units, ReLU	(16)	528
Output Dense	1 unit, linear	(1)	17

Table 11: Companion network (log target, MSE loss)

Layer	Settings	Output shape	Parameters
Input	48 time steps (8 hours) x 15 features	(48, 15)	0
Conv1D	128 filters, kernel size 3, causal padding, ReLU	(48, 128)	5,888
Dropout	rate 0.4	(48, 128)	0
LSTM 1	128 units, tanh, returns the full sequence	(48, 128)	131,584
Dropout	rate 0.4	(48, 128)	0
LSTM 2	128 units, tanh, returns the last step only	(128)	131,584
Dropout	rate 0.4	(128)	0
Dense	16 units, ReLU	(16)	2,064
Output Dense	1 unit, linear	(1)	17

Both networks use the Adam optimizer with a learning rate of 0.003. The main network uses an L2 weight penalty of 0.0001 and a batch size of 256, and the companion network an L2 penalty of 0.00001 and a batch size of 128. The final blend is 0.65 x main + 0.35 x companion.

Reasons for the main choices

- **CNN-LSTM layers.** A small convolution first looks for short local patterns, such as a quick rise in energy use. The LSTM then reads the window in time order and keeps a memory of what happened earlier, which suits a series with a memory of one to a few hours. It had the best validation score of the four architectures (Section 6.1), although the three recurrent types were very close.
- **Small networks.** The main network uses only 32 units and a window of 4 hours. The data is small and noisy, and larger networks did not clearly do better: the best trial of the search (trial 7) had 128 units, but after re-testing with new seeds the small 32-unit network (trial 4) was slightly better (Section 7.2). A dropout sweep showed that a small amount of dropout (0.1) was best and that more dropout made the result worse, which means the small network does not overfit much (Section 7.3).

- **Activation functions.** The LSTM layers use tanh, the standard choice, which keeps values between -1 and 1 and allows fast GPU kernels. The convolution and dense layers use ReLU, which is simple and avoids vanishing gradients. The output layer is linear, because we predict a number.
- **Optimizer: Adam**. It adapts the learning step for each weight and needs little tuning. RMSprop was also tested in the random search; the trials with Adam had a slightly better average score and clearly lower RMSE (Section 7.1).
- **Loss: Huber loss**. It works like squared error for small mistakes but counts large mistakes only in proportion to their size, so a few huge spikes do not dominate learning. Plain mean squared error (MSE) was also tested. MSE is the standard regression loss and matches the RMSE metric, but in our search it gave a higher MAE on average (Section 7.1). The companion network uses MSE.
- **Log target for the companion network.** Energy use is very skewed. Training on log(1 + y) treats quiet and busy periods more evenly, which lowers the typical error (MAE) but raises the error on spikes (RMSE). The main network is trained on raw values and pays more attention to the spikes, and the two networks are blended so that the final model can use both strengths.
- **Regularization.** Dropout after every recurrent layer, a small L2 weight penalty, early stopping on the validation loss, and learning-rate reduction when progress stalls.
- **Training settings.** At most 50 epochs, early stopping with a patience of 8 epochs that restores the best weights, and the learning rate is halved after 3 epochs without progress (minimum 1e-5).

6. Results

All scores below are computed in the original unit (Wh). MAE is the average size of the error. RMSE is the square root of the average squared error, so it punishes large errors more. MAPE is the average error as a percentage of the true value, and R-squared tells how much of the variation of the target the model explains (1 is perfect and 0 is no better than always predicting the average). Choices were made on the validation set only; the test set is used for the final judgement.

Because MAE and RMSE can disagree, decisions use one balanced validation score: MAE and RMSE are each divided by the error of the persistence baseline on the same data, and the two ratios are averaged. A score of 1.0 means "as good as persistence" and lower is better.

6.1 Comparing the architectures

With the same default settings (window 24, 64 units, 2 layers, dropout 0.2, Adam, MSE, raw target) and two seeds each, the four architectures give the results in Table 12. The MLP is clearly the weakest, which shows that modelling the time order helps. The three recurrent models are very close: the balanced scores are 0.891 for the CNN-LSTM, 0.895 for the GRU and 0.898 for the LSTM. These gaps are smaller than the run-to-run variation, so we do not claim that the CNN-LSTM is truly better than the other two. It had the best score and was therefore used in the next steps.

Table 12: Default-setting comparison of the architectures (average of two seeds)

Architecture	Parameters	Val MAE	Val RMSE	Score	Test MAE	Test RMSE
MLP	29,377	28.09	60.85	0.944	27.87	59.74
LSTM	55,617	26.59	58.22	0.898	26.46	58.09
GRU	42,625	26.39	58.27	0.895	26.43	58.22
CNN-LSTM	**71,105**	**26.25**	**57.99**	**0.891**	**26.68**	**58.49**

6.2 Final results and comparison with the baselines

Table 13: Results on the validation and test sets (lower is better for MAE, RMSE and MAPE; higher for R-squared)

Model	Val MAE	Val RMSE	Test MAE	Test RMSE	Test MAPE %	Test R-sq.
Persistence (last value)	27.59	69.93	26.65	66.44	21.9	0.467
Moving average (1 hour)	37.80	83.45	34.30	75.53	28.9	0.312
Linear Regression	27.93	62.64	27.28	59.73	25.2	0.569
Random Forest	25.68	57.27	25.68	58.12	23.1	0.592
Random Forest (log target)	23.76	59.02	23.69	58.88	19.1	0.582
Gradient Boosting	25.32	57.88	25.28	58.83	22.5	0.582
Gradient Boosting (log target)	23.80	58.46	23.64	58.99	19.1	0.580
CNN-LSTM default settings (single run)	25.96	57.52	26.81	58.96	24.3	0.581
CNN-LSTM default settings (5-run average)	24.86	56.88	25.17	57.04	22.7	0.607
CNN-LSTM tuned main model (single run)	24.07	59.00	23.56	57.28	20.0	0.604
CNN-LSTM tuned main model (5-run average)	23.64	58.17	23.43	56.84	20.0	0.610
CNN-LSTM companion model (5-run average)	23.76	59.44	23.24	57.09	19.3	0.607
CNN-LSTM FINAL BLEND (main + companion)	**23.38**	**58.19**	**23.15**	**56.52**	**19.6**	**0.615**

The final blend has the lowest MAE and the lowest RMSE of all models, and the highest R-squared. Its MAPE (19.6%) is lower than that of the baselines trained on raw values, but slightly higher than that of the two log-target tree models (19.1%). Compared with the other models on the test set:

- **Persistence (same as 10 minutes ago):** the final model has 13.1% lower MAE (23.15 against 26.65 Wh) and 14.9% lower RMSE (56.52 against 66.44 Wh).
- **Linear Regression:** MAE 27.28 and RMSE 59.73 Wh. The relationship is not purely linear, but a linear model with good history features is already a reasonable benchmark.
- **Random Forest and Gradient Boosting on raw values:** MAE about 25.3 to 25.7 Wh and RMSE 58.1 to 58.8 Wh. These models are good on RMSE but weaker on MAE.
- **Random Forest and Gradient Boosting on log values:** MAE 23.6 to 23.7 Wh, the best baseline MAE, but RMSE 58.9 to 59.0 Wh. Training on log values lowers the typical error and raises the error on spikes. The final model is 2.1% better on MAE than the best of these (23.64 Wh) and

2.8% better on RMSE than the best baseline RMSE (Random Forest, 58.12 Wh). The strength of the neural networks is that they are good on both metrics at the same time.

- **One CNN-LSTM network, default settings:** a single default run (MAE 26.81, RMSE 58.96 Wh) is not better than the tree models. The average of five default runs reaches an RMSE of 57.04 Wh, which shows how much averaging helps (Section 7.4).

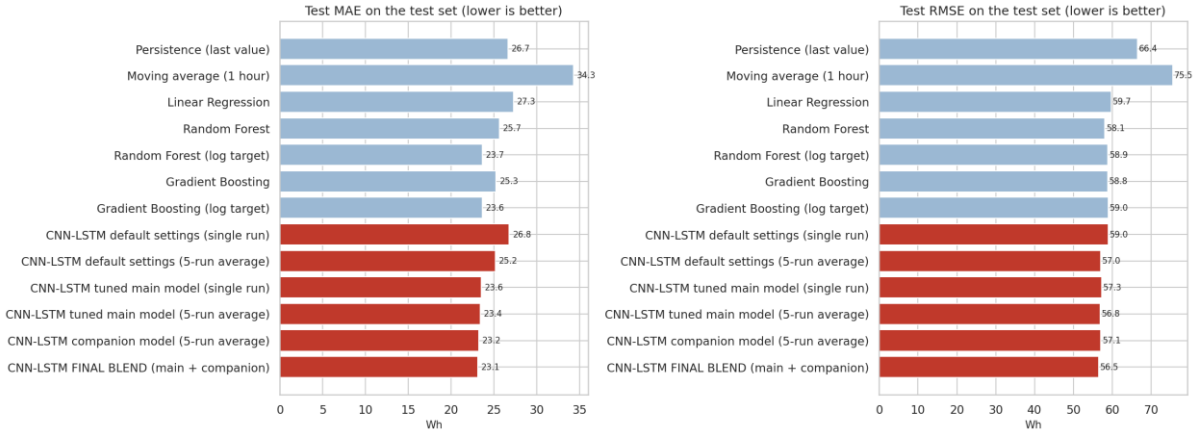


Figure 10: *Test MAE and RMSE of all models. Blue bars are baselines and red bars are deep learning models.*

6.3 Training behavior

Training and validation loss both fall quickly in the first epochs and then flatten (Figure 11). The validation loss is lower than the training loss, which is normal here because dropout is active while training but switched off when the validation loss is computed. After about 15 epochs the training loss keeps falling slowly while the validation loss stays almost flat, and early stopping kept the weights of epoch 23 for the first run of the main network. The validation loss does not rise again, so there is no strong sign of overfitting. The final runs of the main network stopped after 24 to 49 epochs (best epoch 16 to 41) and those of the companion network after 24 to 36 epochs (best epoch 16 to 28). None of the ten final runs reached the limit of 50 epochs, although one run of the main network came close (49 epochs), so a higher limit might give a small extra gain in rare cases.

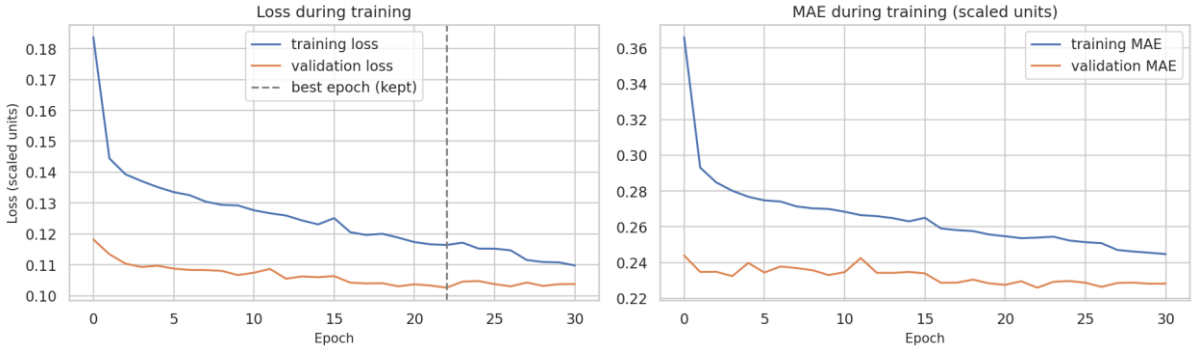


Figure 11: *Training and validation loss (left) and MAE (right) of the main network, first run. Values are in scaled units.*

6.4 Predicted against actual values

Figure 12 shows three days of the test set (top) and the whole test period (bottom). The model follows the normal level of energy use closely and reacts to most of the peaks, but the largest peaks are clearly underestimated. The predicted peaks seem to follow the real peaks by a step or so. This is expected: the model sees the past, so it can only recognize a spike once it has begun and cannot

know exactly when a new one will start. The scatter plots in Figure 13 show the same: the points follow the diagonal for low and medium use and spread out for high use.

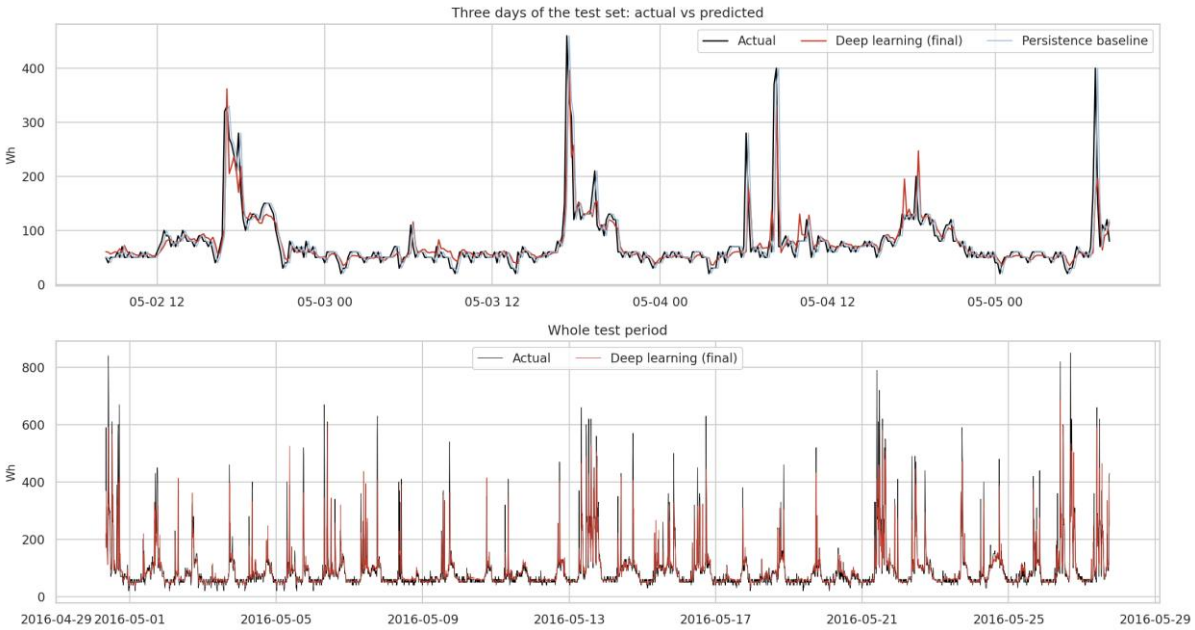


Figure 12: *Actual and predicted energy use on the test set: three days (top) and the whole period (bottom).*

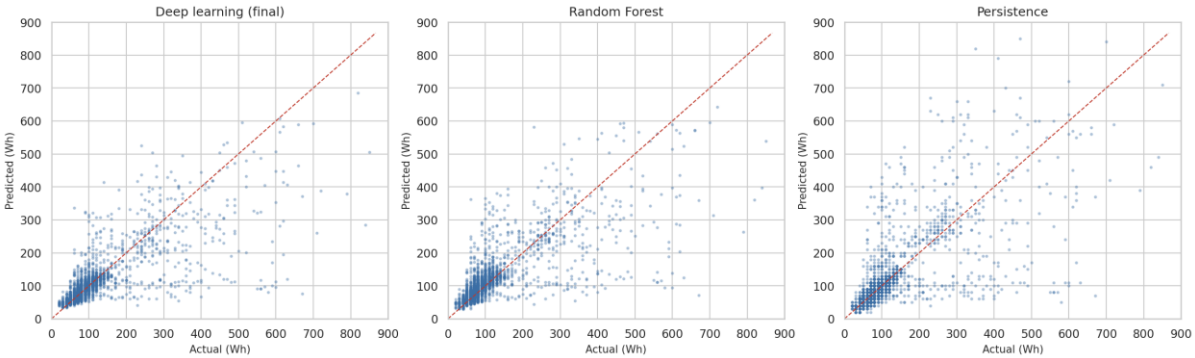


Figure 13: *Predicted against actual values for the final model, the Random Forest and persistence. The dashed line is the perfect prediction.*

6.5 Residual analysis

The residuals (actual minus predicted) are centered close to zero (mean +4.47 Wh, standard deviation 56.34 Wh), so the model has a small overall bias towards under-prediction (Figure 14). The histogram has a very sharp peak with long tails, and the tail on the positive side is longer, which comes from the underestimated spikes. The residual-against-prediction plot shows that the errors grow as the predicted value grows, which means the model is less certain when energy use is high. The residuals over time show no trend. The autocorrelation of the residuals is small at all lags (about 0.06 at most), so little time structure is left in the errors.

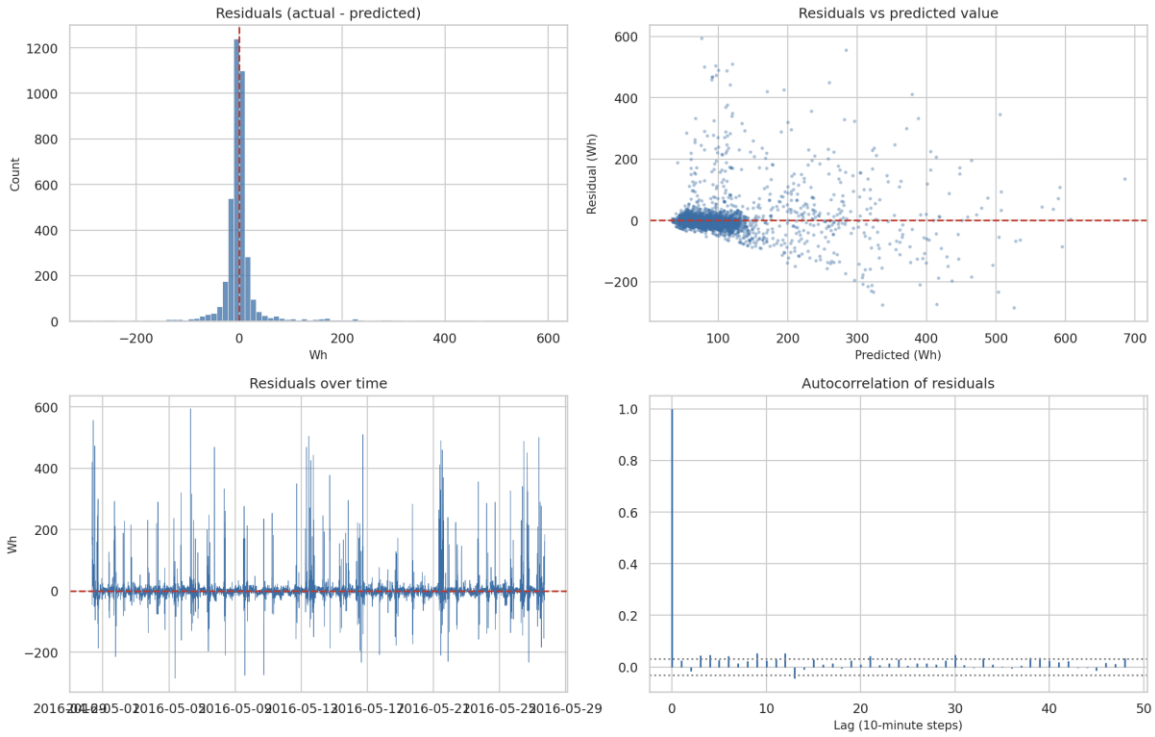


Figure 14: *Residual analysis of the final model on the test set.*

6.6 Where the model works well and where it struggles

Errors are smallest at night (below 10 Wh) and largest around 17:00 and 18:00 (about 60 and 51 Wh) and in the morning around 08:00 (about 43 Wh) (Figure 15). These are the busiest hours. The final model has a lower error than persistence at every hour of the day except 19:00, where it is slightly worse.

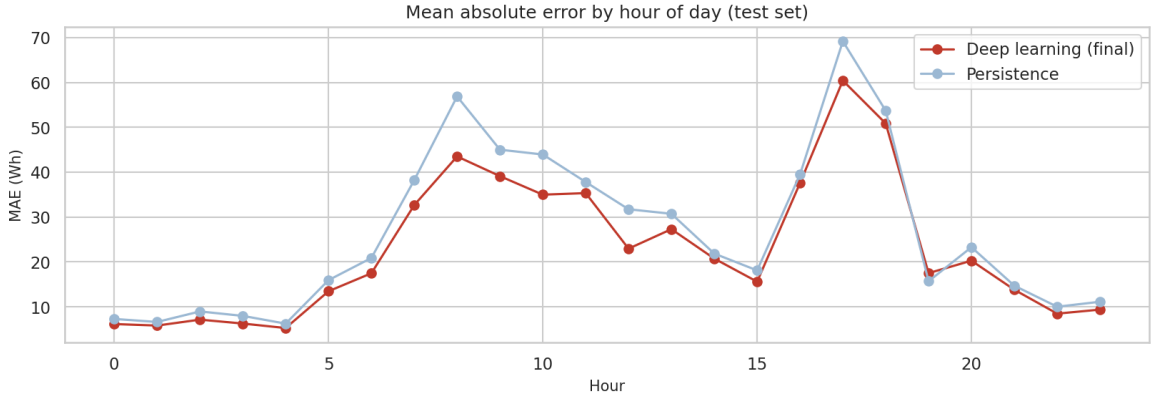


Figure 15: *Mean absolute error by hour of day on the test set.*

Table 14 groups the readings by the true level of energy use. The final model is better than persistence for all levels, including the many quiet readings (79% of the data are 100 Wh or less), and the advantage is largest for readings between 100 and 400 Wh. For the rare readings above 400 Wh it is only about as good as persistence (MAE 223 against 224 Wh), and it **underestimates the biggest peaks**: the average actual value is 535 Wh and the average prediction is 317 Wh. The Huber loss and the log-transformed network reduce the influence of spikes on training, which helps on the quiet readings but costs accuracy on the largest peaks. This is the main limit of the model.

Table 14: Test errors by level of true energy use (final blend)

True use	Readings	MAE of the model	MAE of persistence	Mean actual	Mean predicted
50 Wh or less	1,092	8.62	8.77	47.5	55.6
50 to 100 Wh	2,028	12.40	14.10	72.5	75.8
100 to 200 Wh	502	28.06	37.71	130.0	130.6
200 to 400 Wh	234	95.77	118.25	288.3	230.3
Above 400 Wh	91	223.38	224.29	534.5	317.1

6.7 One-hour-ahead check

The main task is the next 10 minutes. As an extra check, I repeated the pipeline for the energy use **one hour ahead** (6 steps), with the same selected features and the same two network configurations, and one training run each (Table 15). The task is much harder, because the recent past says less about the more distant future. All models get worse, and persistence becomes the worst model (R-squared below zero). The main network and the blend beat all the baselines: the main network has the lowest MAE (34.39 Wh, against 35.83 Wh for the best baseline), the lowest RMSE (77.60 Wh, against 79.99 Wh) and the highest R-squared (0.274), and the blend is almost the same (MAE 34.55 Wh, RMSE 77.62 Wh). The companion network is weaker at this horizon (MAE 36.58 Wh, RMSE 80.97 Wh). The networks were not tuned again for this horizon, and each result is a single run, so it should be read as a quick check and not as the best possible one-hour model.

Table 15: Test results for the one-hour-ahead forecast

Model	MAE	RMSE	MAPE %	R-squared
Persistence (last value)	47.93	104.11	41.1	-0.308
Moving average (1 hour)	44.41	90.29	40.1	0.016
Linear Regression	41.01	81.00	39.3	0.208
Random Forest	41.67	80.28	40.1	0.222
Random Forest (log target)	35.83	79.99	29.1	0.228
Gradient Boosting	41.73	81.65	39.5	0.196
Gradient Boosting (log target)	36.37	80.55	29.6	0.217
CNN-LSTM main model (single run)	34.39	77.60	29.4	0.274
CNN-LSTM companion model (single run)	36.58	80.97	30.2	0.209
CNN-LSTM blend (single runs)	34.55	77.62	29.1	0.273

6.8 Is the result good?

The result is good for this dataset, and it is important to put it in context.

- **Compared with doing nothing smart,** the model is clearly better: MAE is 13.1% lower, RMSE is 14.9% lower and R-squared is 0.615 against 0.467 for persistence. About 61% of the variation of the energy use in the unseen test month is explained.
- **Compared with the other models,** the final blend is the best of all models on MAE, RMSE and R-squared, by about 2% to 3% against the best baseline on each of the two main metrics. Its MAPE (19.6%) is slightly worse than that of the two log-target tree models (19.1%). This is a

small margin, and neural networks vary from run to run by a similar amount for a single run (Section 7.4), which is why the final result is an average of five runs. It is more accurate to say that the networks are good on both metrics at once, while each tree model is good on only one of them.

- **The remaining error is mostly the spikes.** The typical error is about 23 Wh when the median use is 60 Wh, but a reading of 500 Wh or more cannot be foreseen from the past energy use and the indoor sensors alone. The sensor data does not contain what actually causes a spike, for example someone starting the oven.
- **The test is strict.** The split is in time order, the test month is not seen in training, and features use only past data. Results on a random split, which many studies use, would look better because neighboring readings leak into the training set, so they should not be compared directly with ours.

7. Model Optimization

I optimized in steps and compared the choices on the validation set only: a random search over the settings, a re-test of the best candidates with new random seeds, a dropout sweep, and finally averaging and blending. Test results are logged but never used to decide.

7.1 Random search of the hyperparameters

I ran a random search ith 16 trials of the CNN-LSTM. Random search is a good choice because it covers many settings with few training runs, and it works better than a grid when only a few settings really matter. The search space was: window length {6, 12, 24, 36, 48} steps, units per layer {32, 64, 128}, number of recurrent layers {1, 2, 3}, dropout rate {0.1, 0.2, 0.3, 0.4}, L2 penalty {0, 1e-5, 1e-4}, dense units {16, 32, 64}, learning rate {0.0003, 0.001, 0.003}, batch size {64, 128, 256}, optimizer {Adam, RMSprop}, loss {MSE, Huber} and target {raw, log}. Trial 0 is the default design, so the search can never end up worse than where it started, and the target alternates between raw and log so that both are tested equally. The number of epochs was handled by early stopping (maximum 50) instead of being searched.

Table 16: Random search results, best first (validation set; one run per trial)

Trial	Target	Loss	Optim.	Win.	Units	Layers	Drop.	LR	Batch	Val MAE	Val RMSE	Score
T7	log	MSE	Adam	48	128	2	0.4	0.003	128	23.81	60.06	0.861
T4	raw	Huber	Adam	24	32	1	0.3	0.003	256	24.26	59.15	0.863
T11	log	Huber	Adam	48	32	3	0.1	0.003	128	23.88	60.26	0.864
T6	raw	Huber	RMSprop	36	32	2	0.2	0.001	64	24.36	59.64	0.868
T5	log	Huber	RMSprop	6	128	3	0.2	0.001	256	24.10	60.40	0.869
T14	raw	MSE	Adam	24	64	1	0.3	0.001	64	25.37	57.26	0.869
T8	raw	Huber	RMSprop	36	32	2	0.1	0.0003	64	24.61	59.92	0.875
T0	raw	MSE	Adam	24	64	2	0.2	0.001	64	25.68	57.50	0.877
T2	raw	MSE	RMSprop	24	128	3	0.4	0.001	64	25.56	58.01	0.878
T12	raw	MSE	Adam	12	128	2	0.2	0.0003	64	25.53	58.63	0.882
T1	log	MSE	Adam	6	128	2	0.2	0.0003	256	24.83	60.91	0.886
T9	log	Huber	RMSprop	48	64	1	0.4	0.0003	64	24.52	62.68	0.893
T13	log	Huber	RMSprop	24	128	2	0.2	0.0003	256	25.19	61.45	0.896
T15	log	MSE	RMSprop	48	32	1	0.2	0.0003	64	24.82	63.07	0.901
T3	log	MSE	RMSprop	24	64	1	0.4	0.001	256	25.11	63.03	0.906
T10	raw	MSE	RMSprop	24	128	3	0.2	0.0003	256	26.35	59.93	0.906

The factor summary in Table 17 gives a rough guide to what matters (each value is the average over the trials that used it, and the trials also differ in other settings, so this is only an indication). Trials trained on log values or with Huber loss have a lower average MAE (24.5 and 24.4 Wh) but a higher average RMSE (61.5 and 60.5 Wh) than trials on raw values or with MSE loss (MAE 25.2 Wh for both; RMSE 58.8 and 59.8 Wh). In other words, these choices trade RMSE for MAE. Adam and RMSprop have similar MAE, but Adam has the lower RMSE (59.1 against 60.9 Wh).

Table 17: Average validation result by setting (rough guide)

Setting	Value	Trials	Mean val MAE	Mean val RMSE	Mean score
Target	log	8	24.53	61.48	0.884
Target	raw	8	25.22	58.76	0.877
Loss	Huber	7	24.42	60.50	0.875
Loss	MSE	9	25.23	59.82	0.885
Optimizer	Adam	7	24.77	59.11	0.872
Optimizer	RMSprop	9	24.96	60.90	0.888

7.2 Re-testing the best candidates with new seeds

A single run can be lucky, so the two best trials of each target type (raw and log) were trained again with three new seeds (Table 18). The best trial of the search (trial 7, log target, search score 0.861) kept a good score with new seeds (0.863) but was overtaken by trial 4 (raw target), which reached 0.861. Trial 4 became the main network, and trial 7 became the companion network as the best candidate of the other target type. Trials 6 and 11 were slightly worse. In the notebook run the re-test step was executed twice (24 trainings in the log); both executions gave the same winners, trial 4 and trial 7, and Table 18 shows the second execution.

Table 18: Re-test of the best candidates (three new seeds each)

Trial	Target	Loss	Window	Units	Layers	Dropout	Search score	New seeds: mean score	Std
6	raw	Huber	36	32	2	0.2	0.868	0.866	0.004
4	raw	Huber	24	32	1	0.3	0.863	**0.861**	0.006
7	log	MSE	48	128	2	0.4	0.861	**0.863**	0.004
11	log	Huber	48	32	3	0.1	0.864	0.872	0.006

7.3 Dropout sweep

The assessment asks to experiment with different dropout rates, so I trained the main network with dropout rates from 0 to 0.5 (two seeds each, Table 19). A small amount of dropout was best: the score is 0.854 at 0.1, 0.857 at 0.2, 0.864 at 0.3 (the value found by the search), 0.870 with no dropout and 0.875 at 0.5. The rate of 0.1 was better than the 0.3 found by the search by more than 1%, so it replaced it. The curve has a shallow U shape: with no dropout the network overfits a little, and with 0.5 it underfits. Since the best rate is small, the small network does not overfit much, and early stopping together with the small size is enough to control complexity.

Table 19: Dropout sweep for the main network (validation set, average of two seeds)

Dropout rate	Val MAE	Val RMSE	Score	Best epoch
0.0	24.79	58.88	0.870	17.0
0.1	24.15	58.20	0.854	24.5
0.2	24.20	58.51	0.857	26.5
0.3	24.29	59.31	0.864	31.0
0.5	24.39	60.60	0.875	39.0

7.4 Early stopping, averaging and blending

Early stopping on the validation loss was used in every training run, with a patience of 8 epochs and the best weights restored.

Neural networks start from random weights, so runs with the same settings give slightly different results. The five final runs of the main network gave test MAE values between 23.56 and 24.26 Wh and test RMSE values between 57.21 and 57.95 Wh. The five runs of the companion network gave MAE values from 23.29 to 24.04 Wh and RMSE values from 57.44 to 58.25 Wh. To reduce this randomness, each network in the final model is the **average of five runs**. Averaging lowered the RMSE of the main network from 57.28 Wh (single run) to 56.84 Wh and its MAE from 23.56 to 23.43 Wh.

Blending. The main and the companion network are combined as w x main + (1 - w) x companion, with w chosen on the validation set from 0 to 1 in steps of 0.05. The validation score is almost flat between w = 0.55 and w = 0.80 (0.8398 to 0.8408) and is best at w = 0.65 (0.840), against 0.844 for the main network alone (w = 1) and 0.856 for the companion network alone (w = 0). So the choice of the weight does not matter much. The reason is that the two networks make very similar mistakes: the correlation of their test errors is 0.969. Blending two models helps most when their errors are

different, so a large gain was not expected, and the small gain that we see (Table 21) should be read with care.

Table 20: Blend weight and validation score (lower is better)

Weight of the main network	0 (companion only)	0.25	0.50	0.65 (chosen)	0.80	1 (main only)
Validation score	0.856	0.846	0.841	0.840	0.841	0.844

7.5 Effect of the optimization

Table 21 separates the effect of each step. The default model is the CNN-LSTM with default settings, trained with the same five seeds as the final models.

Table 21: Before and after optimization (test set unless marked)

Step	Val MAE	Val RMSE	Test MAE	Test RMSE	Test MAPE %	Test R-sq.
1. Default settings, one run	25.96	57.52	26.81	58.96	24.3	0.581
2. Default settings, 5-run average	24.86	56.88	25.17	57.04	22.7	0.607
3. Tuned main network, one run	24.07	59.00	23.56	57.28	20.0	0.604
4. Tuned main network, 5-run average	23.64	58.17	23.43	56.84	20.0	0.610
5. Final blend (main + companion)	23.38	58.19	23.15	56.52	19.6	0.615

What each step did:

- **Tuning the settings (step 1 to 3, and step 2 to 4).** The test MAE falls by 12.1% for a single run (26.81 to 23.56 Wh) and by 6.9% for the five-run average (25.17 to 23.43 Wh), while the test MAPE falls from about 22.7% to 20.0%. This comes mainly from the Huber loss (the search showed that Huber loss lowers the MAE) and from the small network. The RMSE improves only a little: a single tuned run is better than a single default run (58.96 to 57.28 Wh, 2.8%), and the five-run averages are almost equal (57.04 and 56.84 Wh, 0.4%).
- **Averaging five runs (step 1 to 2, and step 3 to 4).** The test RMSE falls by 3.2% for the default settings (58.96 to 57.04 Wh) and by 0.8% for the tuned network (57.28 to 56.84 Wh), and the MAE falls as well (by 6.1% and 0.5%). Averaging helps the default network a lot and the tuned network only a little, because the tuned network is already more stable. Averaging mainly reduces the randomness of a single run.
- **Blending (step 4 to 5).** MAE falls from 23.43 to 23.15 Wh (1.2%) and RMSE from 56.84 to 56.52 Wh (0.6%). This is a small gain, similar in size to the run-to-run variation, and it is consistent on both metrics on the test set (on the validation set the MAE improves and the RMSE stays the same). I keep it because it does no harm, but the report does not claim that blending is a major improvement.
- In total (step 1 to 5), the test MAE falls by 13.7% (26.81 to 23.15 Wh), the RMSE by 4.1% (58.96 to 56.52 Wh) and the MAPE from 24.3% to 19.6%, while R-squared rises from 0.581 to 0.615.

8. Challenges and Solutions

Table 22: Main difficulties and how they were handled

Challenge	What happened	Solution
Dataset differs from the brief	NSM, WeekStatus and Day_of_week are missing; there are extra columns.	Created the three features from the date, kept the extra columns and documented the difference.
MAE and RMSE point in different directions	Training on log values or with Huber loss lowers MAE but raises RMSE; squared-error training does the opposite.	Used one balanced validation score (each error relative to persistence), tested both target types, and averaged and blended networks of both types.
Skewed target with spikes	A few very large readings dominate RMSE and are hard to predict.	Kept the real spikes, tested Huber loss and a log target, and reported errors by consumption level.
Avoiding data leakage	Lags, rolling windows, scaling and outlier limits can all leak future information.	Split by the time of the target, fitted every scaler and limit on training data only, left a gap before validation, and added an automatic leakage test.
Capping removed real information	Percentile limits capped about 22,000 test values, because May is warmer than the training months.	Switched to wider 3 x IQR limits learned from training data; only 0.17% of values are touched.
No history at the start for the typical-use feature	The first days of each day type had no earlier days to average, which left gaps in the time line.	Used the average over all day types as a fallback and dropped the first day, so the time line stays continuous.
Lucky winners in the search	The best trial of the search (trial 7) lost its first place to trial 4 after re-testing with new seeds.	Re-tested the best candidates with three new seeds, used different seeds for the final models, and averaged five runs.
Weak sensor signal and limited data	Only 4.5 months from one house; sensors add little beyond past energy use.	Used strict time-ordered evaluation, a noise benchmark for feature selection and honest reporting of small gains.

9. Conclusion

9.1 Key findings

- The recent history of energy use is by far the most useful information for a 10-minute forecast. Raw sensors alone cannot predict the target (negative R-squared), and the 15 selected features are all history or time-of-day features.
- Feature selection with a random-noise benchmark and RFE reduced 93 candidates to 15 and improved the validation score compared with using all features (0.874 against 0.924).
- The final model, a blend of two CNN-LSTM networks (each an average of five runs), reached MAE 23.15 Wh, RMSE 56.52 Wh, MAPE 19.6% and R-squared 0.615 on a strict, time-ordered test set. It has the lowest MAE and RMSE of all models and is 13.1% and 14.9% better than persistence on the two metrics.
- Tuning the settings (including Huber loss) mainly lowers MAE and MAPE, averaging five runs mainly lowers RMSE, and blending adds only a small extra gain because the two networks make very similar errors (correlation 0.969).

- The model follows normal use well, but it underestimates the largest peaks (above 400 Wh), and its predictions of a spike appear after the spike has started.

9.2 Limitations

- The data covers 4.5 months of one house, and the test set is a single month. A walk-forward evaluation with several test periods would give a stronger estimate.
- The margins over the best baselines are small (about 2% to 3%), and a statistical test over repeated runs was not done.
- A few training runs came close to or reached the 50-epoch limit (one final run stopped after 49 epochs, and one re-test run reached 50), so a higher limit might give a small extra gain.
- The holiday list assumes Belgian public holidays, and only one holiday falls in the training period.
- The one-hour model reuses the settings tuned for 10 minutes ahead, with one run each.

10. References

- Candanedo, L. M., Feldheim, V. and Deramaix, D. (2017) 'Data driven prediction models of energy use of appliances in a low-energy house', *Energy and Buildings*, 140, pp. 81-97.
- Cho, K. et al. (2014) 'Learning phrase representations using RNN encoder-decoder for statistical machine translation', *Proceedings of EMNLP 2014*, pp. 1724-1734.
- Chollet, F. (2021) *Deep Learning with Python*. 2nd edn. Shelter Island, NY: Manning Publications.
- Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735-1780.
- Huber, P. J. (1964) 'Robust estimation of a location parameter', *The Annals of Mathematical Statistics*, 35(1), pp. 73-101.
- Srivastava, N. et al. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15, pp. 1929-1958.
- TensorFlow (no date) *Time series forecasting (tutorial)*. Available at: https://www.tensorflow.org/tutorials/structured_data/time_series

Appendix A. Code, Environment and Reproducibility

Repository: https://github.com/Piratheepan22/Appliance_Energy_Forecasting . All code is in one notebook (notebooks/Appliance_Energy_Forecasting.ipynb), organised in numbered sections with functions and docstrings. The README.md explains how to run it in Google Colab or on a local machine.

Table 23: Environment of the reported run

Item	Value
Platform	Google Colab with a GPU runtime
TensorFlow / Keras	2.21.0 / 3.13.2
NumPy / pandas	2.1.3 / 2.2.3
scikit-learn	1.6.1
Matplotlib / Seaborn / joblib	3.10.0 / 0.13.2 / 1.6.0
Random seeds	Final runs: 42, 7, 2024, 11 and 99. Architecture comparison: 1 and 2. Re-test: 1, 2 and 3. Dropout sweep: 1 and 2. Random search: 42.
Networks trained	73 (8 architecture, 16 search, 24 re-test, 10 dropout sweep, 15 final), about 54 minutes of training time in total

Table 24: Final configuration

Setting	Main network	Companion network
Architecture	CNN-LSTM: Conv1D (32 filters) + 1 LSTM layer (32 units)	CNN-LSTM: Conv1D (128 filters) + 2 LSTM layers (128 units)
Window	24 steps (4 hours)	48 steps (8 hours)
Target	Raw Appliances, standardized	log(1 + Appliances), standardized
Loss	Huber	MSE
Optimizer / learning rate	Adam / 0.003	Adam / 0.003
Dropout / L2	0.1 / 0.0001	0.4 / 0.00001
Dense units / batch size	16 / 256	16 / 128
Parameters	10,337	271,137
Early stopping	Patience 8 on validation loss	Patience 8 on validation loss
Final model	Average of 5 runs, blend weight 0.65	Average of 5 runs, blend weight 0.35
Input features	15 selected features (Table 8)	15 selected features (Table 8)